LAB 5
STATIC CODE ANALYSIS

Retail Point of Sale & Discount Tracker

Student	Dhanush Gowda A S
SRN	PES1UG25CS812
Repository	SE-Labs-PES1UG25CS812
Lab folder	LAB5_PES1UG25CS812

1. Objective

The objective of this lab is to examine a Retail POS starter program using Pylint, Bandit and Flake8, document code-quality and security weaknesses, remediate the implementation, and compare analyzer output and application behaviour before and after the changes.

2. Tools Used

Tool	Purpose
Pylint	Python code quality, design diagnostics, naming and documentation checks.
Bandit	Security-oriented static analysis for Python source code.
Flake8	Style and lint checks using PyFlakes, pycodestyle and McCabe.
unittest	Regression tests for checkout, input validation, promotions, inventory updates and receipt logging.

3. Implementation Summary

The original implementation accepted arbitrary promo expressions through **eval()**, silently ignored checkout errors, reused a mutable default receipt list, accepted invalid quantities and tax rates, failed to update inventory, and left receipt files unmanaged. The corrected implementation replaces these behaviours with an allow-list of promo codes, explicit validation, controlled state, real stock updates, and context-managed receipt logging.

The corrected application supports the promo codes **SAVE10**, **SAVE50** and **CLEAR**. Unknown promo expressions are rejected instead of executed.

2. Static Analysis Metrics (Before vs After)

Tool	Focus Area	Before Refactoring	After Refactoring	Status
Bandit	Application security (AST scan)	3 issues (2 Low, 1 Medium)	**0 issues**	**PASS**
Flake8	PEP 8 style, whitespace, line length	11 reported violations	**0 violations**	**PASS**
Pylint	Code quality, design, docstrings	5.78 / 10.00	**10.00 / 10.00**	**PASS**
unittest	Functional regression coverage	Baseline defects present	9 regression tests pass	**PASS**

3. Identified Issues & Remediation Log

Issue / Code	Tool / Type	Severity	Description	Fix Approach
B307: eval	Bandit	Medium	User-controlled promo text could execute arbitrary Python expressions.	Replaced eval() with allow-listed SAVE10, SAVE50 and CLEAR codes.
B311: random	Bandit	Low	Pseudo-random receipt identifier generation is not suitable for security-sensitive use.	Removed random receipt identifier generation; use sequential receipt numbering.
B110: try/except/pass	Bandit	Low	Stock update errors were swallowed without informing the caller.	Added explicit product and quantity validation with meaningful exceptions.
W0102: mutable default	Pylint	Medium	receipts=[] was shared across calls and could leak state.	Changed default to None and append only to a supplied list.
W0702 / W0718: broad exceptions	Pylint / Flake8	High	Checkout and stock failures were hidden or silently ignored.	Replaced broad handlers with explicit validation and ValueError/TypeError.
Input validation	Logic	High	Unknown products, non-positive quantities and invalid tax rates were accepted or mishandled.	Validate product names, integer positive quantities and tax rate 0–1.
Inventory update	Logic	Medium	The starter reassigned a price to itself and did not change stock.	Added a stock dictionary and incremented the requested product quantity.
Receipt file handling	Pylint	Medium	Receipt file was opened without deterministic close or explicit encoding.	Used a context manager and UTF-8 encoding.
PEP 8 / docstrings	Pylint / Flake8	Low	Missing docstrings, naming violations, unused import and formatting issues.	Added module/function docstrings, snake_case names and clean formatting.

4. Output Comparison (Before vs After)

The examples below are derived from the original starter program and the corrected implementation. They show how invalid inputs and unsafe promo expressions are handled after remediation.

Scenario	Before Fixes	After Fixes
Valid checkout	3 pens + 2 notebooks: subtotal Rs. 120.00; with 5% tax, Rs. 126.00.	Same valid cart produces Rs. 126.00; the total is recorded in the caller-provided receipt list.
Unknown product	Unknown item is silently ignored by the broad exception handler; checkout continues with a partial bill.	Raises ValueError: Unknown product. The invalid cart is not silently billed.
Negative quantity	4 bottles at quantity -4 produces a negative subtotal and a misleading negative bill.	Raises ValueError because quantity must be greater than zero.
Invalid quantity / tax	String quantities can be swallowed as a failed cart item; a negative tax rate is not rejected.	Raises TypeError for non-integer quantity and ValueError for tax rates outside 0–1.
Promo expression	eval('total * 0.9') executes the expression; arbitrary user input is evaluated as Python.	SAVE10 applies a 10% discount; SAVE50 subtracts Rs. 50 (not below zero); CLEAR leaves the amount unchanged. Unknown expressions are rejected.
Stock update	update_stock('pen', 5) prints a message but leaves stock unchanged.	Starting pen stock 100 becomes 105 after update_stock('pen', 5).
Receipt logging	File handle is not explicitly closed and encoding is unspecified.	Receipt is appended using a context-managed file handle with UTF-8 encoding.

5. Validation and Reproducibility

The regression suite contains nine tests covering checkout totals and receipts, unknown products, non-positive quantities, invalid tax rates, string quantities, supported promo codes, rejected promo expressions, stock updates and receipt logging. The test suite and analyzer outputs were generated during the earlier GitHub Actions execution; the analysis workflow has since been removed from the repository. No claim is made that a new workflow is currently active.

To reproduce locally from the LAB5_PES1UG25CS812 directory:

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_analysis.py
```

6. Deliverables

The Lab 5 folder contains the original baseline source, corrected POS source, issue register, reflection, README, analysis runner, requirements, regression tests, six before/after analyzer output files, and this consolidated PDF report.

Report note

Metrics shown here are taken from the committed analyzer output files: Pylint 5.78/10 before and 10.00/10 after; Bandit 3 findings (2 Low, 1 Medium) before and none after; Flake8 11 reported violations before and none after. This report is a project report and does not replace any separate official lab handout or instructor template.